🧠 1. What is NumPy?

NumPy stands for **Numerical Python**.
It is a **Python library** used for **fast mathematical and scientific computations**.

It provides:

- Powerful **multi-dimensional arrays (ndarray)**
- Mathematical operations on arrays (vectorized operations)
- Linear algebra, random numbers, Fourier transform, etc.

🚀 2. Why do we need NumPy?

Python lists are flexible but **slow** for numerical operations.
NumPy arrays are:

- **Faster** (implemented in C)
- **Memory-efficient**
- **Support element-wise operations**

Task	Python List	NumPy Array
Storage	More memory	Compact memory
Speed	Slow	Very fast
Operations	Loops required	Vectorized (no loops)
Data type	Mixed	Homogeneous

⚙️ 3. Installation and Import

pip install numpy

Then import it:

import numpy as np

📦 4. Creating Arrays

a) From Python List

```
import numpy as np


arr = np.array([1, 2, 3, 4, 5])

print(arr)

print(type(arr))
```

b) Multi-dimensional Array

```
arr2 = np.array([[1, 2, 3], [4, 5, 6]])

print(arr2)
```

🔍 5. Array Attributes

```
print(arr2.ndim)   # Number of dimensions

print(arr2.shape)  # (rows, columns)

print(arr2.size)   # Total elements

print(arr2.dtype)  # Data type of elements
```

🧩 6. Useful Array Creation Functions

```
np.zeros((2,3))       # 2x3 array filled with 0

np.ones((3,2))        # 3x2 array filled with 1

np.arange(0,10,2)     # [0, 2, 4, 6, 8]

np.linspace(0,1,5)    # [0., 0.25, 0.5, 0.75, 1.]

np.eye(3)             # 3x3 identity matrix
```

➕ 7. Basic Operations

Let's create two arrays:

```
a = np.array([1, 2, 3])
```

b = np.array([4, 5, 6])

Operation	Code	Result
Addition	a + b	[5 7 9]
Subtraction	a - b	[-3 -3 -3]
Multiplication	a * b	[4 10 18]
Division	b / a	[4. 2.5 2.]
Power	a ** 2	[1 4 9]

📊 8. Mathematical Functions

NumPy provides built-in mathematical functions:

```
np.sqrt(a)       # Square root
np.exp(a)        # Exponential
np.log(a)        # Natural log
np.sum(a)        # Sum of elements
np.mean(a)       # Average
np.max(a)        # Max element
np.min(a)        # Min element
```

🎯 9. Indexing and Slicing

```
arr = np.array([10, 20, 30, 40, 50])
print(arr[0])    # 10
print(arr[1:4])  # [20 30 40]
print(arr[-1])   # 50
```

For 2D arrays:

```
arr2 = np.array([[10,20,30],[40,50,60]])
```

```
print(arr2[0,1])  # 20
print(arr2[:,1])  # [20 50]  (2nd column)
```

🔁 10. Reshaping and Flattening

```
arr = np.arange(1, 7)
arr_reshaped = arr.reshape(2, 3)
print(arr_reshaped)
```

Output:

```
[[1 2 3]
 [4 5 6]]
```

Flatten:

```
arr_flat = arr_reshaped.flatten()
print(arr_flat)   # [1 2 3 4 5 6]
```

🎲 11. Random Numbers

```
np.random.rand(2,3)   # 2x3 array of random floats (0–1)
np.random.randint(1,10,5)  # 5 random integers from 1 to 9
```

🧮 12. Matrix Operations

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print(np.dot(A, B))   # Matrix multiplication
print(np.transpose(A))
print(np.linalg.inv(A))   # Inverse
```

✅ 13. Real-Life Use Cases

Domain	Use
Data Science	Fast data manipulation
Machine Learning	Matrix calculations
Image Processing	Pixel array operations
Engineering	Linear algebra, simulation
Finance	Statistical computations

🎓 Summary

Concept	Description
Array	Core data structure in NumPy
ndarray	N-dimensional homogeneous array
Vectorized Operations	Perform element-wise operations without loops
Speed	Much faster than Python lists
Integration	Works with pandas, scikit-learn, TensorFlow, etc.